

#used to rectify the errors after the process of cleaning etc.

```
import pandas as pd
import seaborn as sns
```

```
#Load titanic dataset
df=sns.load_dataset("titanic")
```

```
print(df.head())
print(df.shape)
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class \
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

(891, 15)

```
df.info()
```

```

class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    object
8   class       891 non-null    category
9   who         891 non-null    object
10  adult_male  891 non-null    bool
11  deck        203 non-null    category
12  embark_town 889 non-null    object
13  alive       891 non-null    object
14  alone       891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB

```

1.Inconsistent formats

Concept

The same val **is** represented **in** diff formats.ex:India,INDIA,Indias same word butt diff **format** so detect **and** rectify

Create Problems

```

df.loc[0,'embark_town']='southampton'
df.loc[1,'embark_town']='SOUTHAMPTON'
df.loc[2,'embark_town']='Southampton' #row,col change these 3 val and print all so first print these and bal
print(df['embark_town'].unique())

```

2. Duplicate Rows

A duplicate row **is** an exact copy of another row

Create duplicate records

```
duplicate_rows=df.iloc[:5]
df_dup=pd.concat([df,duplicate_rows]) #we create dup
print(df_dup.shape)
```

(896, 15)

#Find duplicates

```
print(df_dup.duplicated().sum())
```

112

View Duplicate Rows

```
df_dup[df_dup.duplicated()]
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class \
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Southampton	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

```
['southampton' 'SOUTHAMPTON' 'Southampton' 'Queenstown' 'Cherbourg' nan]
```

```
: #It change all words first letter in capital and balance small letter  
df['embark_town']=df['embark_town'].str.title()  
print(df['embark_town'].unique()) #next fill the na using fillna
```

```
['Southampton' 'Queenstown' 'Cherbourg' nan]
```

```
: print(df['embark_town'].isnull().sum()) #total nll val
```

```
2
```

```
: df['embark_town']=df['embark_town'].fillna('America')  
print(df['embark_town']) #fill the val
```

```
0      Southampton
```

```
1      Southampton
```

```
2      Southampton
```

```
3      Southampton
```

```
4      Southampton
```

```
...
```

```
886     Southampton
```

```
887     Southampton
```

```
888     Southampton
```

```
889      Cherbourg
```

```
890      Queenstown
```

```
Name: embark_town, Length: 891, dtype: object
```

```
: print(df['embark_town'].isnull().sum()) #sum gives 0
```

```
0
```

```
: print(df['embark_town'].unique()) #it adds america
```

Remove duplicates

```
df_dup=df_dup.drop_duplicates()  
print(df_dup.duplicated().sum())
```

0

Typos

Spelling mistakes create unnecessary categories.

create **type**

```
df.loc[0,'embark_town']='Southamton'  
df.loc[1,'embark_town']='Southhampton' #we create mistakes  
  
print(df['embark_town'].value_counts()) #mistake words also add
```

```
embark_town  
Southampton      643  
Cherbourg         167  
Queenstown        77  
America           2  
Southamton        1  
Southhampton      1  
Name: count, dtype: int64
```

#Fix types

```
df['embark_town']=df['embark_town'].replace({  
    'Southamton':'Southampton',  
    'Southhampton':'Southampton'  
})
```

#Verify

```
print(df['embark_town'].value_counts()) #back to form
```

```
embark_town
Southampton    645
Cherbourg      167
Queenstown     77
America        2
Name: count, dtype: int64
```

```
print(df['embark_town'].unique()) #again print previous val
```

```
['Southampton' 'Queenstown' 'Cherbourg' 'America']
```

```
#Detect Possible Types-sorted types-dlt null val and print only unique val
```

```
print(sorted(df['embark_town'].dropna().unique()))
```

```
['America', 'Cherbourg', 'Queenstown', 'Southampton']
```

4.Unit Mismatches

Values representing the same measurement stored in diff units

Create Problems

Suppose some ages are accidentally entered in months instead of years.

```
df.loc[0,'age']=240
```

```
df.loc[1,'age']=300
```

```
#Detect Unusual values
```

```
df['age'].describe()
```



```
#Detect Unusual values
```

```
df['age'].describe()
```

```
count    714.000000
mean      30.371387
std       19.362012
min        0.420000
25%       20.125000
50%       28.000000
75%       38.750000
max      300.000000
Name: age, dtype: float64
```

```
#Fix Unit Mismatch
```

```
#Convert months to years
```

```
df.loc[df['age']>120,'age']=(
    df.loc[df['age']>120,'age']/12 #above 120 div by 12 and convert months to yrs.
)
#Verify
df['age'].describe()
```

```
count    714.000000
mean      29.678109
std       14.525902
min        0.420000
25%       20.000000
50%       28.000000
75%       38.000000
max       80.000000
Name: age, dtype: float64
```

```
5.Wrong category labels
```

```
df['survived']
```

```
Contains
```

```
0
```

```
1
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class \
0	0	3	male	20.0	1	0	7.2500	S	Third
1	1	1	female	25.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone	survived_label
0	man	True	NaN	Southampton	no	False	No
1	woman	False	C	Southampton	yes	False	Yes
2	woman	False	NaN	Southampton	yes	True	Yes
3	woman	False	C	Southampton	yes	False	Yes
4	man	True	NaN	Southampton	no	True	No

4 man True Southampton no True No

```
] : Handling Null Values in titanic dataset  
what are null values?  
null val occurs when data is unavailable
```

```
df.isnull().sum()
```

```
] : survived            0  
pclass                0  
sex                    0  
age                    0  
sibsp                  0  
parch                  0  
fare                   0  
embarked              2  
class                  0  
who                    0  
adult_male            0  
embark_town           0  
alive                  0  
alone                  0  
survived_label        0  
dtype: int64
```

```
] : #Method 1:Mean imputation  
#Replace missing values with the avg value.  
df['age']=df['age'].fillna(df['age'].mean())  
df.head(20)
```

```
#check:
df.isnull().mean() #deck has most null val so dlt
```

```
survived      0.000000
pclass        0.000000
sex           0.000000
age           0.198653
sibsp         0.000000
parch         0.000000
fare          0.000000
embarked      0.002245
class         0.000000
who           0.000000
adult_male    0.000000
deck          0.772166
embark_town   0.000000
alive         0.000000
alone         0.000000
survived_label 0.000000
dtype: float64
```

```
df.drop(columns=['deck'], inplace=True, errors='ignore')
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	20.0	1	0	7.2500	S	Third	
1	1	1	female	25.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	embark_town	alive	alone	survived_label
0	man	True	Southampton	no	False	No
1	woman	False	Southampton	yes	False	Yes
2	woman	False	Southampton	yes	True	Yes
3	woman	False	Southampton	yes	False	Yes

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone	survived_label
0	0	3	male	20.000000	1	0	7.2500	S	Third	man	True	Southampton	no	False	No
1	1	1	female	25.000000	1	0	71.2833	C	First	woman	False	Southampton	yes	False	Yes
2	1	3	female	26.000000	0	0	7.9250	S	Third	woman	False	Southampton	yes	True	Yes
3	1	1	female	35.000000	1	0	53.1000	S	First	woman	False	Southampton	yes	False	Yes
4	0	3	male	35.000000	0	0	8.0500	S	Third	man	True	Southampton	no	True	No
5	0	3	male	29.678109	0	0	8.4583	Q	Third	man	True	Queenstown	no	True	No
6	0	1	male	54.000000	0	0	51.8625	S	First	man	True	Southampton	no	True	No
7	0	3	male	2.000000	3	1	21.0750	S	Third	child	False	Southampton	no	False	No
8	1	3	female	27.000000	0	2	11.1333	S	Third	woman	False	Southampton	yes	False	Yes
9	1	2	female	14.000000	1	0	30.0708	C	Second	child	False	Cherbourg	yes	False	Yes
10	1	3	female	4.000000	1	1	16.7000	S	Third	child	False	Southampton	yes	False	Yes
11	1	1	female	58.000000	0	0	26.5500	S	First	woman	False	Southampton	yes	True	Yes
12	0	3	male	20.000000	0	0	8.0500	S	Third	man	True	Southampton	no	True	No
13	0	3	male	39.000000	1	5	31.2750	S	Third	man	True	Southampton	no	False	No
14	0	3	female	14.000000	0	0	7.8542	S	Third	child	False	Southampton	no	True	No
15	1	2	female	55.000000	0	0	16.0000	S	Second	woman	False	Southampton	yes	True	Yes
16	0	3	male	2.000000	4	1	29.1250	Q	Third	child	False	Queenstown	no	False	No
17	1	2	male	29.678109	0	0	13.0000	S	Second	man	True	Southampton	yes	True	Yes
18	0	3	female	31.000000	1	0	18.0000	S	Third	woman	False	Southampton	no	False	No

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone	survived_label
0	0	3	male	20.000000	1	0	7.2500	S	Third	man	True	Southampton	no	False	No
1	1	1	female	25.000000	1	0	71.2833	C	First	woman	False	Southampton	yes	False	Yes
2	1	3	female	26.000000	0	0	7.9250	S	Third	woman	False	Southampton	yes	True	Yes
3	1	1	female	35.000000	1	0	53.1000	S	First	woman	False	Southampton	yes	False	Yes
4	0	3	male	35.000000	0	0	8.0500	S	Third	man	True	Southampton	no	True	No
5	0	3	male	29.678109	0	0	8.4583	Q	Third	man	True	Queenstown	no	True	No
6	0	1	male	54.000000	0	0	51.8625	S	First	man	True	Southampton	no	True	No
7	0	3	male	2.000000	3	1	21.0750	S	Third	child	False	Southampton	no	False	No
8	1	3	female	27.000000	0	2	11.1333	S	Third	woman	False	Southampton	yes	False	Yes
9	1	2	female	14.000000	1	0	30.0708	C	Second	child	False	Cherbourg	yes	False	Yes
10	1	3	female	4.000000	1	1	16.7000	S	Third	child	False	Southampton	yes	False	Yes
11	1	1	female	58.000000	0	0	26.5500	S	First	woman	False	Southampton	yes	True	Yes
12	0	3	male	20.000000	0	0	8.0500	S	Third	man	True	Southampton	no	True	No
13	0	3	male	39.000000	1	5	31.2750	S	Third	man	True	Southampton	no	False	No
14	0	3	female	14.000000	0	0	7.8542	S	Third	child	False	Southampton	no	True	No
15	1	2	female	55.000000	0	0	16.0000	S	Second	woman	False	Southampton	yes	True	Yes
16	0	3	male	2.000000	4	1	29.1250	Q	Third	child	False	Queenstown	no	False	No

Lets check Age distribution.

```
#df['age'].skew()
```

If skewness is high:

Use median

If data is nearly normal:

Use mean

#Method 3:Mode imputation

#Replace missing values with the most frequent value.

```
df['embarked_mode']=df['embarked'].fillna(df['embarked'].mode()[0])
```

```
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone	survived_label	embarked_mode
0	0	3	male	20.0	1	0	7.2500	S	Third	man	True	Southampton	no	False	No	S
1	1	1	female	25.0	1	0	71.2833	C	First	woman	False	Southampton	yes	False	Yes	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Southampton	yes	True	Yes	S
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	Southampton	yes	False	Yes	S
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Southampton	no	True	No	S

#Method 4:Forward Fill(FFill)

Use the previous val to fill missing data

Before imputation

#apply forward fill-change using previous values

```
df['embarked_ffill']=df['embarked'].ffill()
```

#Check missing values

```
print(df['embarked_ffill'].isnull().sum())
```

0

```
df.columns #embarked ffill added
```

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',  
      'embarked', 'class', 'who', 'adult_male', 'embark_town', 'alive',  
      'alone', 'survived_label', 'embarked_mode', 'embarked_ffill'],  
      dtype='object')
```

Method 5:KNN Imputation

K-Nearest neighbours(KNN) Imputation fills missing values using info from similar rows

Instead of using a single value(mean/median),it finds the K most similar records and uses their values.

```
import pandas as pd
```

```
import numpy as np
```

```
data={  
    'age':[22,25,np.nan,24,23],  
    'fare':[7.25,8.05,7.90,8.00,7.50]  
}
```

```
df=pd.DataFrame(data)
```

```
print(df)
```

```
#Check missing values
import pandas as pd
import seaborn as sns
from sklearn.impute import KNNImputer

#Load Titanic Dataset
df=sns.load_dataset('titanic')

#Forward fill for embarked
df['embarked_ffill']=df['embarked'].ffill()

#KNN Imputation for age
cols=[
    'age',
    'fare',
    'sibsp',
    'parch',
    'pclass'
]

imputer=KNNImputer(n_neighbors=5)

df[cols]=imputer.fit_transform(df[cols])

print(df.isnull().sum())
```


	age	fare
0	22.0	7.25
1	25.0	8.05
2	NaN	7.90
3	24.0	8.00
4	23.0	7.50

#Apply KNN Imputation

```
from sklearn.impute import KNNImputer
KNN=KNNImputer(n_neighbors=3)
```

```
df_KNN=pd.DataFrame(
    KNN.fit_transform(df),
    columns=df.columns)
print(df_KNN)
```

#sklearn -> Scikit-Learn library for machine learning and data preprocessing.

#impute -> Module for handling missing values.

#KNNImputer -> Class that fills missing values using the K-Nearest Neighbors(KNN).

#fit_Transform(df) -> Learns from the data and fills missing values.

#Converts the result back into a Pandas DataFrame.

	age	fare
0	22.0	7.25
1	25.0	8.05
2	24.0	7.90
3	24.0	8.00
4	23.0	7.50

```
survived      0
pclass        0
sex           0
age           0
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town    2
alive         0
alone         0
embarked_ffill 0
dtype: int64
```

```
] : Backward fill
df.fillna(method='bfill', inplace=True)

or
df.bfill(inplace=True)
```

Encoding Using Titanic Dataset

```
] : What is Encoding?
    Ml models work with numbers, not text.
ex:
Sex
male
female
male
```

A model cannot directly understand "male" and "female".
We convert categories into numbers using encoding.

Types of Encoding

```
# Label encoding(original data)
Assigns a number to each category.
Example:
Size
    .Small
    .Medium
    .Large
becomes
Size
    .0
    .1
    .2
```

When to use Label encoding?

Ordinal categories

ex:

Small < medium < large

Low < medium < high

Poor < avg < good

Nominal Categories

Ex:

Red,Blue,Green

Male,female

India,USA,UK